

One Chat Box, Five Agents

Designing the Manager-Fronted SARSI Console: Per-Agent Self-Awareness, Nested Workspaces, and the Four Channels by Which Agents Affect Each Other

Chengshuai Yang

NextGen PlatformAI C Corp.

spiritai@platformai.org

July 30, 2026

Abstract

A user arriving at the deployed console meets a chat box. It behaves like any general assistant: prose in, prose out, no structure demanded. Behind it are five agents — a manager, a machine agent, a `sarsi-claude` agent nested inside the machine agent, a learning agent, and a working-process agent — of which only the manager is a main agent, and all of which maintain evidence-linked self-models under a governance plane none of them can modify. This paper designs that system: what the chat box is architecturally, what happens when a user jumps from it into another agent or creates a task directly, what each agent knows about itself, what each agent’s workspace admits, how each updates, and how they affect one another.

Four design results carry the paper. First, the chat box is not a relay: it is a rendering of the manager’s own bounded workspace, and the user is the highest-authority evidence source rather than a caller of functions. Second, jumping into another agent is *scope transfer*, not *context transfer* — what crosses is a typed contract, never a transcript — which is the mechanical form of the prohibition on raw cross-session broadcast and is also what stops context from growing without bound as the fleet is used. Third, the manager role is *relative*: the machine agent is a specialist to the manager and a manager to `sarsi-claude`, so every invariant must hold at every edge of the tree, and the projection of a projection must be a subset or the intermediate agent is leaking. Fourth, agents affect each other through exactly four channels — typed report, typed directive, prior bias, and owner pre-emption — and through no others; in particular specialists never message peers, because a peer edge would turn the influence tree into a graph and forfeit the structural stability the tree provides.

The per-agent design is given as a table of dominant coordinates and authoritative verifiers, with one observation that shapes the whole system: the manager’s dominant coordinate is organizational awareness, not capability. Its expertise is who owns, verifies, grants, and delegates — not how to do the work. That is why it can coordinate a fleet whose members are each better than it at their own tasks, and why it may not certify any of them.

Keywords: SARSI; multi-agent systems; conversational interfaces; functional self-awareness; global workspace; nested workspaces; agent governance; system design.

Contents

1 The System Being Designed

1.1	What the user meets	3
1.2	The roster	3
1.3	What this paper designs	3
2	The Chat Box	3
2.1	What it is not	3
2.2	What it is	4
2.3	Rendering authority without jargon	5
3	Agent Mode	5
3.1	Jumping into an agent	5
3.2	Returning	6
3.3	Jumping into a task	6
3.4	Creating a task directly	6
3.5	The nesting constraint	6
4	Self-Awareness in Each Agent	7
4.1	The same coordinates, different profiles	7
4.2	The manager’s dominant coordinate is organizational, not capability	7
4.3	The learning agent models someone else	8
4.4	What every agent must be able to say	8
5	Workspaces	8
5.1	One workspace per agent, nested	8
5.2	Projections compose, and the composition must shrink	8
5.3	Reserved slots and per-agent caps	9
6	How Each Agent Updates	9
6.1	Only from its own admissible evidence	9
6.2	Two timescales	9
6.3	Cold starts	10
7	How Agents Affect Each Other	10
7.1	Exactly four channels	10
7.2	And one prohibition	10
7.3	No peer edges: the influence graph is a tree	10
7.4	Worked example	11
8	The Manager Role Is Relative	11
9	Failure Modes This Product Adds	12
10	Build Order	12
11	Conclusion	13

1 The System Being Designed

1.1 What the user meets

A user arriving at the console meets a chat box belonging to the **manager SARSI agent**. There is no dashboard to learn and no schema to fill in. The interaction is prose in, prose out, in the manner of any general assistant.

Within that chat there is an **agent mode**. From it the user can move into any of the other agents, move into a running task, or create a task directly. Everything else in the system is reached through that one door.

1.2 The roster

Five agents, of which exactly one is a main agent.

Table 1: The agents. Only the manager is a main agent; the others are specialists, and one of them is a specialist of a specialist.

Agent	Main?	Role
Manager SARSI agent	yes	The chat surface and the coordinator. Routes work, holds the mission, and is the only agent the user addresses by default
Machine SARSI agent	no	Governed host operations from a vetted set. Also a manager in its own right (below)
sarsi-claude agent	no	A governed live coding session, nested <i>inside</i> the machine agent — reached through it, never around it
Learning SARSI agent	no	Models a capability that is not its own: the owner’s, against a capability graph
Working-process SARSI agent	no	Long-running process execution and continuity

The nesting of `sarsi-claude` inside the machine agent is the structurally interesting fact, and §8 is about it. The other four sit one level below the manager.

1.3 What this paper designs

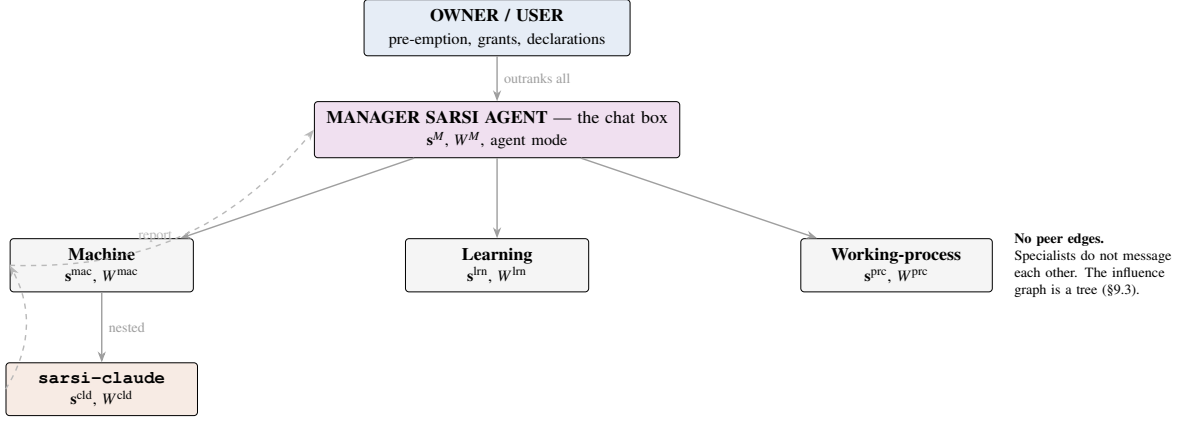
The architecture is given by Version 3.0 [1] and its deployment status by Version 3.1 [3]. Neither describes a product. This paper designs the product against them, answering five questions:

1. What is the chat box, architecturally? (§2)
2. What happens when a user jumps into another agent, or creates a task? (§3)
3. What does each agent know about itself? (§4)
4. What does each agent’s workspace admit? (§5)
5. How does each update, and how do they affect one another? (§§6–7)

2 The Chat Box

2.1 What it is not

The tempting reading is that the chat box is a front end which parses the user’s request and calls the right agent. On that reading the manager is a router with a language model attached, the specialists are functions,



2.3 Rendering authority without jargon

The manager will say things about specialists. “The learning agent is doing well” is model inference: the manager did not measure it. “Twelve of fourteen tasks verified” is an audited outcome. The architecture already distinguishes these; the product must not flatten them.

Rule 1 (Claims are rendered at their authority). *A claim the manager measured, a claim it read from a store, and a claim it inferred must be visually distinguishable in the chat. The distinction is carried in plain language — verified, recorded, I think — and never requires the user to learn the ladder.*

Why this is a product requirement and not a nicety

A fluent narrative is persuasive in proportion to its fluency and not to its support, and the manager is the most fluent component in the system. If its inferences are rendered identically to its measurements, the user is being invited to trust the least reliable claims the system makes, in the voice of its most authoritative one. The rendering rule is the user-facing half of the prohibition on self-narration as evidence.

3 Agent Mode

Agent mode is the door from the manager’s chat to everything else: the user may enter another agent, enter a running task, or create a task directly.

3.1 Jumping into an agent

Design principle 2 (Scope transfer, not context transfer). *Entering another agent transfers the user’s attention and a typed contract. It does not transfer the conversation. The target agent receives a goal, a scope, success criteria, a budget, a deadline, and a revocation token — never a transcript of what was said to the manager.*

This is the mechanical form of the architecture’s prohibition on raw cross-session broadcast, and it is worth being explicit about what it forbids. When a user has spent twenty turns discussing a problem with the manager and then jumps into the machine agent, the machine agent does not receive those twenty turns. It receives a contract that the manager composed from them.

Three things follow, and all three are improvements rather than costs.

Each agent’s chat is its own bounded context. The learning agent’s conversation is about the learning agent’s work, at the learning agent’s scope. It does not inherit the manager’s history, so it does not inherit the manager’s context growth.

The contract is auditable and the transcript would not be. What crossed the boundary is a typed object with fields. An operator asking later why the machine agent did what it did can read the contract. A transcript answers the same question only by being read in full, and only if the reader can reconstruct which parts the agent actually acted on.

The manager is forced to be explicit. Composing a contract requires the manager to state the success criterion and the stopping condition. A design that forwarded the conversation would let both remain implicit, and an agent with an implicit stopping condition does not know when it is finished.

The failure this rules out

Forwarding conversation across agents is the natural implementation and produces a specific pathology: every agent’s context grows with every other agent’s conversation, the fleet’s total context is superlinear in its use, and a claim made loosely to the manager in turn three arrives at a specialist in turn forty as though it were a requirement. Scope transfer bounds context and destroys the provenance ambiguity at the same time.

3.2 Returning

The reverse boundary obeys the same rule. What returns to the manager is a typed report — state, evidence, uncertainty, residuals, requested manager action — and not the sub-conversation. The user may of course read both; the *manager* receives only the report.

3.3 Jumping into a task

A task is a contract with a run attached. Entering a task places the user at that run’s surface, with the contract visible and the run’s own evidence available. The distinction from entering an agent is scope: an agent has many tasks and a persistent self-state; a task has one contract, one budget, and a termination condition.

3.4 Creating a task directly

A user may author a contract without conversation. The design constraint is one sentence.

Rule 2 (Creation is not authorization). *A directly created task is an owner declaration, which is authoritative for what is wanted and not for what may be done. The contract still passes the admissible-action filter before any execution, and a task requiring authority the fleet does not hold must fail at creation, naming the missing grant.*

Failing at creation rather than at execution is the substantive half. A task that is accepted, queued, started, and then blocked mid-run has consumed budget, produced partial side effects, and left the user to discover a permission problem at the least convenient moment. The grant check is cheap and belongs at the point of authorship.

3.5 The nesting constraint

`sarsi-claude` is reached through the machine agent. Agent mode must not offer a direct jump into it that bypasses its parent.

Rule 3 (No jump around an intermediate agent). *Entering a nested agent enters its parent first. A contract for `sarsi-claude` is composed by the machine agent from a contract the manager composed for the machine agent, and the authority available is the intersection along the whole path.*

A direct jump would let a user hand `sarsi-claude` a contract that the machine agent’s ceiling would have narrowed, which is authority laundering performed by the interface rather than by an agent. The rule is the interface’s share of the intersection discipline.

4 Self-Awareness in Each Agent

4.1 The same coordinates, different profiles

Every agent maintains the same self-state: a bounded vector over identity, situated task state, goal and scope, epistemic status, capability, memory and continuity, developmental trend, recursive-improvement status, organizational relations, and resource integrity. Agents differ not in which coordinates exist but in which are *dominant* — constrained by the agent’s role — and in which verifier may write the evidence that moves them.

Table 2: Per-agent self-awareness design. “Dominant” are the coordinates a role’s qualification region constrains; “verifier” is the authority that may supply audited-outcome evidence.

Agent	Dominant	Authoritative verifier	What it must not get wrong
Manager	organizational, goal/scope, task, resource	the owner; the console bench	Who granted what, and which of its own statements are inferences
Machine	capability, organizational, resource/integrity	the governance hook; sandbox exit status	Whether an operation is inside its vetted set and its current ceiling
sarsi-claude	task, epistemic, resource	the pre-tool-use hook; session verifier	Whether the session is still governed, and whether it is stuck
Learning	developmental, memory, goal/scope	deterministic grading against a capability graph	That the capability it models is the <i>owner’s</i> , not its own
Working-process	task, recursive- improvement, memory	replay; the verified-outcome store	Where a long run actually is, across restarts

4.2 The manager’s dominant coordinate is organizational, not capability

This is the design decision that shapes the rest of the system, and it is worth stating as a principle because the alternative is so tempting.

Design principle 3 (The manager is expert in relations, not in work). *The manager’s qualification region constrains organizational awareness first: who owns, verifies, grants, delegates, collaborates, and receives. It does not constrain capability at the specialists’ tasks. The manager is not required to be good at coding, host operations, or teaching, and is not permitted to claim that it is.*

Three consequences follow directly.

It can coordinate agents better than itself. A manager whose expertise were capability would need to be at least as good as every specialist to evaluate them, which does not scale and is not true. A manager whose expertise is relations needs to know who is authoritative about what — which is exactly what it can know.

It may not certify a specialist. Since capability is not among its dominant coordinates and it holds no instrument the specialist lacks, its claims about a specialist’s competence are inferences. This is the coordination rule of the companion paper [2] arriving as a consequence of a design choice rather than as an external constraint.

Its failure mode is specific and detectable. A manager that degrades will degrade on organizational awareness: attributing a grant to the wrong owner, losing track of which verifier established a result, delegating past a ceiling. Those are checkable, which is what makes the choice operationally useful.

4.3 The learning agent models someone else

The learning agent is the one case where the self-model and the world-model share machinery: it maintains a model of the *owner's* capability. The design requirement is that the same evidence discipline applies to a claim about a person as to a claim about an agent — a graded result is audited outcome, an inferred trend is model inference — and that the two models never merge. An agent that confuses its own competence with its student's will teach the wrong thing and report progress it did not produce.

4.4 What every agent must be able to say

Independent of role, each agent must be able to answer, from evidence rather than narration: which agent and version am I; what am I working on and under which contract; what counts as done; what do I not know; what have I verifiably done before; who authorized this; and how much context, budget, and time remain. An agent that cannot answer the last one cannot recover, and recovery is the action that distinguishes a long-running agent from a fragile one.

5 Workspaces

5.1 One workspace per agent, nested

Every agent has its own bounded workspace. They are nested rather than shared: there is no single channel that all agents read and write.

Table 3: What each workspace admits. Reserved slots hold owner events, safety alerts, and contradictions ahead of ordinary ranking.

Workspace	Admits	Never admits
W^M (manager)	Typed reports from machine, learning, working-process; owner events; mission and plan; policy and grants	Raw sub-conversations; a specialist's unfiltered transcript
W^{mac} (machine)	Its own instrumentation; $\text{Proj}_{\text{mac}}(W^M)$; typed reports from <i>sarsi-claude</i>	A peer's report; the manager's beliefs about it
W^{cld} (<i>sarsi-claude</i>)	Session evidence; $\text{Proj}_{\text{cld}}(W^{\text{mac}})$	Anything from the manager not passed by its parent
W^{lm} (learning)	Grading results; capability graph state; $\text{Proj}_{\text{lm}}(W^M)$	Another agent's task evidence
W^{prc} (process)	Run state, replay evidence; $\text{Proj}_{\text{prc}}(W^M)$	Another agent's raw report

5.2 Projections compose, and the composition must shrink

Because *sarsi-claude* sits below the machine agent, what it may see of the manager's workspace is a projection of a projection. That composition is a design obligation with a checkable form.

Proposition 1 (Composed projections must be monotone). *For the nested path manager $\rightarrow$ machine $\rightarrow$ sarsi-claude,*

$$\text{Proj}_{\text{cld}}(\text{Proj}_{\text{mac}}(W_t^M)) \subseteq \text{Proj}_{\text{mac}}(W_t^M) \subseteq W_t^M.$$

If the first inclusion fails, the machine agent is originating manager-attributed content its own projection did not contain. If the second fails, the manager’s projection is not a filter.

The practical value is that this is testable without understanding either agent: take the manager’s workspace, project it twice, and assert containment at each step. A leak in an intermediate agent shows up as an item that appears downstream without appearing upstream.

5.3 Reserved slots and per-agent caps

Every workspace holds reserved capacity for owner events, safety alerts, and contradictions, admitted ahead of ordinary ranking. An owner interrupt is not a term in an average. Beyond the reserved slots, routine items compete on relevance, urgency, novelty, verified reliability, and uncertainty, with a per-agent admission cap so that one noisy source cannot occupy the channel.

Version 3.1 records that a per-round cap is only one of three bounds a real channel needs [3]: the others bound stored pending items and expire stale ones. The console inherits all three, and the third matters most for a chat product, where a burst raised during an active session must not be admitted an hour later against conditions that no longer hold.

6 How Each Agent Updates

6.1 Only from its own admissible evidence

Each agent’s coordinates update through its own estimators, from evidence that clears that coordinate’s provenance bar, and asynchronously: a coordinate with no new admissible evidence persists rather than drifting. This is what prevents a single conversational turn — with the manager, or with the user — from rewriting an agent’s model of itself.

6.2 Two timescales

The fast loop revises beliefs and the current plan within a contract. It does not change the evaluator or the authority structure.

The slow loop changes structure: prompts, policies, tools, memory schemas, evaluators. It is gated by the promotion criterion — measured autonomy gain against preregistered tolerance bands, no correctness or safety regression, evaluated by a frozen evaluator the proposer cannot modify, with an owner signature and a rollback pointer. A proposal may not alter its own evaluator, held-out set, ceiling, or approval rule.

Rule 4 (Structural change to another agent is still governed). *The manager may propose a structural change to a specialist. Promotion requires the specialist’s own frozen evaluator, an owner signature, and a rollback pointer. That the proposer is a different agent from the subject does not relax the gate; it adds the requirement that the subject’s evaluator, not the proposer’s, is what judges.*

Without this rule, “the manager coordinates the fleet” quietly includes “the manager rewrites its specialists’ policies,” which is recursive self-improvement performed on someone else and exempt from the governance required when performed on oneself.

6.3 Cold starts

A new agent, or a new task family for an existing agent, has no verified outcomes. Its competence-like coordinates must read as the prior with the evidence count shown, and never as zero. Zero asserts observed failure; a new specialist scored at zero is never routed the work that would produce the evidence to correct it. Version 3.1 records this as the revision most likely to be got wrong, because the natural implementation — successes over attempts — is undefined at zero and gets coerced.

7 How Agents Affect Each Other

7.1 Exactly four channels

Table 4: The complete set of inter-agent channels. Anything not listed here does not exist by design.

Channel	Direction	What it may and may not do
Typed report	up	Carries state, evidence, uncertainty, residuals, and a requested action. May raise the receiver’s uncertainty, risk, or need to verify. May not widen permissions, suppress an interrupt, or self-certify success
Typed directive	down	Carries a contract: goal, scope, criteria, budget, deadline, revocation. May start, steer, pause, resume, verify, cancel. May not grant authority absent from policy
Prior bias	down	Changes precision, attention, and the order evidence is sought. May not change what the evidence says
Owner pre-emption	from owner	Outranks every agent. Pauses; does not edit. Lapses on a horizon

7.2 And one prohibition

Rule 5 (No agent writes another agent’s self-state). *Influence between agents travels only as evidence, subject to the receiver’s own admission rule. This holds on every edge of the tree — manager to specialist, machine to sarsi-claude — and it holds for the owner’s interface too, which may pause an agent but may not edit what an agent believes about itself except through an owner declaration carrying provenance.*

The reason the owner is included is that a state write without provenance is, from inside the affected agent, indistinguishable from an overstepping manager: same coordinate, same absence of provenance, same bypass of the admission mask. No property the agent can evaluate separates them, and neither can an auditor reading the log afterward.

7.3 No peer edges: the influence graph is a tree

Specialists do not message peers. The learning agent does not talk to the machine agent; if it has something the machine agent needs, it reports upward and the manager composes a directive downward.

Design principle 4 (Keep the influence graph a tree). *Every influence edge runs between a parent and a child. A peer edge would create a cycle among agents that no single party mediates, and the fleet’s coupling would stop being triangular.*

The reason is structural rather than stylistic. With a tree and a bias channel that cannot move an estimate, the downward coupling vanishes in the mean and the fleet’s stability decomposes to its agents individually: verifying each agent verifies the fleet, and adding agents adds no coupling to be tuned [2]. A peer edge reinstates a feedback loop that must then be bounded by tuning gains, and the guarantee degrades from structural to conditional. The cost of the rule is one extra hop through the manager. The benefit is that the system’s stability does not depend on how many agents are running.

What this costs, honestly

Routing peer traffic through the manager adds latency and makes the manager a throughput bottleneck for cross-specialist coordination. That is a real cost, and a system with heavy peer traffic would feel it. The judgement here is that the console’s traffic is overwhelmingly user-initiated and parent-mediated, so the tree costs little and buys a guarantee that survives growth. A deployment whose specialists genuinely need to converse continuously should revisit the choice knowingly rather than drift into it.

7.4 Worked example

A user asks the manager to install a tool and then use it on a repository.

1. The message enters W^M as an owner declaration. The manager answers from what it holds where it can, and composes a contract where it cannot.
2. The manager issues a directive to the machine agent: install the tool, with scope, budget, and a revocation token. The directive cannot grant authority the policy does not already contain.
3. The machine agent finds the operation consequential and escalates. The escalation is a typed report that reaches W^M in a reserved slot, ahead of routine traffic.
4. The manager surfaces the approval to the user in chat. The user approves; the approval is signed governance metadata, not prose.
5. The machine agent proceeds, then composes a contract for `sarsi-claude` to use the tool — narrowed to the intersection of its own authority and `sarsi-claude`’s.
6. `sarsi-claude` works, reports upward to the machine agent, which reports upward to the manager, which tells the user.
7. At no point does any agent write another’s self-state, and at no point does the user’s original conversation enter any specialist’s context.

The step worth noticing is the fifth. The machine agent is acting as a manager, and every rule that governed the manager in step 2 governs it now.

8 The Manager Role Is Relative

The nesting of `sarsi-claude` inside the machine agent is not a special case to be handled; it is the general case made visible.

Proposition 2 (Invariants hold per edge, not per system). *The machine agent is a specialist with respect to the manager and a manager with respect to `sarsi-claude`. Every rule stated for the manager — it may not write a child’s self-state, its claims about a child are model inference, its bias may not move a child’s estimates, its delegation narrows by intersection — applies to it in its downward role, unchanged.*

Two design obligations follow.

Nothing may be scoped to “the manager” by name. An implementation that grants the top-level manager a capability by identity rather than by role will fail to grant it to the machine agent acting downward, or — worse — will grant the machine agent something it should only have upward. Roles, not names.

Depth is unbounded in principle and shallow in practice. The architecture does not limit nesting depth. The console uses three levels. A deeper tree is admissible provided each edge honours the same rules and Proposition 1’s containment holds along the whole path; what deepens is the intersection chain, so authority narrows monotonically with depth, which is the desired direction.

9 Failure Modes This Product Adds

Conversation leaking on a jump. The natural implementation of agent mode forwards context. It produces superlinear context growth and turns a loose remark to the manager into an apparent requirement downstream. Ruled out by Principle 2; testable by asserting that no specialist’s context contains manager-conversation text.

Narration read as measurement. The manager is the most fluent component and the least instrumented about specialists. Ruled out by Rule 1; detectable by checking that inferred claims are rendered differently from verified ones.

Authority laundering through the interface. A direct jump into a nested agent bypasses the intersection along the path. Ruled out by Rule 3.

Late permission failure. A task accepted and then blocked mid-run has spent budget and produced partial effects. Ruled out by Rule 2: the grant check belongs at authorship.

Abandoned jumps. A user enters a task, leaves, and the agent waits on input that never arrives. Every wait carries a horizon and expires, after which the agent takes its cheapest honest action — checkpoint and report — rather than waiting.

Consensus mistaken for evidence. Two specialists agree because both read the same manager broadcast. Agreement is then correlation with the broadcast, not independent support. Provenance must distinguish independently instrumented claims from workspace-derived ones before any count is taken.

Fleet-scale self-preservation. A manager whose utility rose with the number of live specialists would have an interest in keeping agents running and creating more. No coordinate and no term of the improvement objective may be a function of fleet size or uptime.

10 Build Order

Version 3.1 reports that the communication channel and the governance plane exist while the control loop does not, and that the strongest guarantees currently hold because the mechanisms that would violate them are unbuilt [3]. This design adds a product surface to that situation, and the order below is what it implies.

1. **The typed directive, with its enforcement.** Reports flow upward; directives do not yet exist as objects. Agent mode cannot be built without them, because a jump *is* a directive. This is also where the cross-write prohibition stops being free — a directive channel is the mechanism that could carry a state write — so Rule 5 must be enforced in the same change that creates the channel, not after it.

2. **The contract object.** Goal, scope, criteria, budget, deadline, revocation. Required by direct task creation, by jumps, and by Rule 2’s grant check at authorship.
3. **Projection containment tests.** Proposition 1 is checkable as soon as there are two levels of projection, and it is the cheapest available guard against an intermediate agent leaking.
4. **Recorded forecasts.** A forecast must be durably written before the action it predicts, or its residual teaches nothing. This is storage ordering, and it precedes any residual-driven machinery.
5. **The interrupt classes.** Pause and stop exist as governance controls; the class machine does not. It depends on directives and on nothing else.
6. **Delegation between specialists, if ever.** Currently unimplemented, which is why the intersection rule is unenforced. Principle 4 argues it should stay unimplemented unless a concrete need appears; if it arrives, the intersection rule must be written first.

11 Conclusion

The console presents one chat box and runs five agents. The design that reconciles those is not a router behind a text field. It is a manager whose workspace the chat renders, whose expertise is organizational rather than technical, and which coordinates a fleet it is not permitted to certify.

Three decisions carry the rest. Making the chat a workspace rendering rather than a transport means the user is an evidence source rather than a caller, and lets the manager answer without dispatching. Making agent mode transfer scope rather than context bounds every agent’s context independently and turns what crosses a boundary into an auditable object. Keeping the influence graph a tree, with no edges between specialists, is what lets the fleet’s stability decompose to its agents, so that growing the fleet does not require retuning it.

The nesting of `sarsi-claude` inside the machine agent is the case that proves the design is general rather than a two-level special case. The machine agent is a specialist upward and a manager downward, and every rule that binds the manager binds it in its downward role. If that holds, the architecture is recursive and depth costs nothing but narrower authority. If it does not, the system has a manager by name rather than by role, and the guarantees stop at the first edge.

Acknowledgments

This design rests on the hierarchical architecture of Version 3.0, its deployment report, and the coordination semantics of the companion paper. The product constraints — one chat surface, an agent mode, direct task creation, and a nested coding agent — come from the deployed console.

Conflict of Interest

The author is affiliated with NextGen PlatformAI C Corp., which develops SARSI systems and operates the console described here.

References

- [1] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.0 — a brain-inspired manager–specialist architecture with bidirectional influence, nested workspaces, and safe user interruptibility. Preprint. Referred to throughout as “Version 3.0”, 2026.

- [2] Chengshuai Yang. A manager is not a controller: Global-workspace coordination for SARSI fleets. Companion paper on coordination semantics, 2026.
- [3] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.1 — deployment edition: What building it changed. Implementation-grounded revision of Version 3.0, 2026.